

Guia completo do Checkpoint 2

Programação Distribuída — Aluno 1 e Aluno 2

Referência: código local em 30/09/2026. Este guia descreve a implementação atual, inclusive as alterações locais ainda não enviadas ao Git. Os resultados de teste indicados como históricos vêm dos relatórios existentes; a seção de verificação registra separadamente o que foi executado para este guia.

1. O que o checkpoint entrega

O sistema compartilha documentos com dois processos: o Peer armazena e transfere os bytes; o Super Peer registra quem possui cada chunk. O usuário usa bin/peer como CLI. Upload e download são executados pelo serviço Peer já iniciado, com seu NodeID cadastrado. O Super Peer responde à busca, mas não armazena o arquivo.

Responsabilidades do enunciado: Aluno 1 cuida de upload, download, armazenamento, fragmentação, SHA-256, LZ4 e transferência paralela. Aluno 2 cuida de metadados, hash table, ObjectID e registro dos chunks. O checkpoint 1 fornece TCP, framing, CRC32, NodeID e JOIN/LEAVE; esses componentes continuam no caminho do checkpoint 2.

```

Usuário / CLI
| comando local por socket Unix
v
Peer ativo ----- LOOKUP / ANNOUNCE -----> Super Peer
| índice: ObjectID -> descritores -> NodeIDs
| STORE / DOWNLOAD_REQ
v
Peer de armazenamento: pending / objects
  
```

2. Arquivos e responsabilidades

- Aluno 1 — peer.c: CLI, servidor de armazenamento, JOIN, anúncio, atendimento de STORE e DOWNLOAD_REQ. Reúne o antigo peer_service.c.
- Aluno 1 — file_client.c: orquestra upload/download e workers paralelos; calcula e confere hashes e métricas.
- Aluno 1 — storage.c: manifests, chunks locais, retomada, commit e verificação no disco.
- Aluno 1 — compression.c, content.c: adaptação ao LZ4, SHA-256 de buffers, nome e extensão .pdf.
- Aluno 2 — node.c: configuração e NodeID = SHA-256(IP binário || porta em ordem de rede || UUID).
- Aluno 2 — superpeer.c: tabela de membros, JOIN/LEAVE, servidor TCP, ANNOUNCE/LOOKUP e main. Reúne os antigos membership.c e superpeer_app.c.
- Aluno 2 — metadata.c, metadata.h: ObjectID, FileMetadata, hash table e disponibilidades dos chunks.
- Integração — directory.c: converte dados de transferência em metadados e resolve NodeID para IP/porta durante LOOKUP.
- Compartilhados — network.c, protocol.c, transfer_protocol.c, rpc.c, concurrent_server.c, local_control.c, app_config.c: sockets, framing, serialização, chamadas, concorrência, canal local e identidade persistente.

bin/client é um link para bin/peer; bin/node é um link para bin/superpeer. Esses aliases mantêm comandos anteriores, sem criar processos diferentes.

3. Identidade, mensagens e dados

NodeID tem 32 bytes e identifica um nó. `node_compute_id` usa SHA-256 de IP normalizado em bytes, porta e UUID. `app_identity` persiste o UUID em `node.uuid` para manter a identidade ao reiniciar com o mesmo IP anunciado e porta. `superpeer_register_node` mantém membros em vetor protegido por mutex; `superpeer_find_member` entrega uma cópia para a camada de diretório.

ObjectID também tem 32 bytes, mas identifica o conteúdo completo: SHA-256(arquivo). `object_id_file` lê o arquivo em blocos e atualiza o digest sem carregá-lo inteiro. Dois nomes para bytes iguais geram o mesmo ObjectID; uma mudança nos bytes altera o ID.

O header TCP tem 98 bytes: versão, tipo, NodeID de origem/destino, TransactionID, timestamp, tamanho do payload e CRC32. `protocol_send_message` serializa campos explicitamente; `protocol_receive_message` lê header e payload completos e confere o CRC32. `transfer_protocol.c` serializa os payloads de documento, chunk, ANNOUNCE e LOOKUP com inteiros em ordem de rede. Ponteiros de structs C não são enviados. `rpc_call` abre a conexão e confere a resposta e seu TransactionID.

Um chunk cobre no máximo 4 MiB = 4.194.304 bytes originais. Cada chunk tem índice, offset, tamanho original, tamanho comprimido, SHA-256 do conteúdo original e bytes LZ4. O limite do payload TCP é 5 MiB. Os códigos `M_STORE` e `M_DOWNLOAD_REP` identificam o tipo geral; o primeiro byte do payload identifica a operação específica. No upload são usados `STORE/BEGIN`, `STORE/CHUNK`, `STORE/COMMIT` e, depois, `STORE/ANNOUNCE`.

4. Modelo do Aluno 2

O cabeçalho `metadata.h` usa exatamente a estrutura pedida no trabalho:

```
typedef struct {
    uint8_t object_id[32];
    char filename[256];
    uint64_t size;
    uint32_t chunk_count;
    uint8_t **chunk_hashes;
    uint32_t version;
    uint32_t owner;
} FileMetadata;
```

`object_id` é o SHA-256 do documento; `chunk_hashes[i]` é o SHA-256 do chunk descomprimido. `version` começa em 1. `owner` é um resumo numérico dos quatro primeiros bytes do NodeID; as localizações conservam o NodeID completo, pois 32 bits não bastam como identidade única. `MetadataChunk` guarda índice, offset e tamanhos; LZ4 fica no documento de transferência, fora da estrutura exigida.

`MetadataStore` tem 257 buckets. `bucket` escolhe o bucket usando os 32 bytes do ObjectID; colisões usam lista encadeada e comparação do ID completo. O mutex protege buscas, inclusão e remoção. A tabela é local ao Super Peer, em memória; não é uma DHT nem faz lookup distribuído.

O anúncio percorre `superpeer.c` (`register_announcement`) → `directory.c` (`directory_announce`) → `metadata.c` (`metadata_announce`). A função valida os descritores, prepara a entrada completa e só então troca o registro na tabela. Cada chunk recebe uma associação (índice, NodeID do Peer). Um anúncio repetido compatível preserva localizações anteriores e acrescenta o novo Peer; conflito de conteúdo com o mesmo ObjectID é rejeitado.

`metadata_find_document` devolve cópia independente, incluindo hashes; o chamador usa `file_metadata_free`. `metadata_find_name` localiza o ObjectID pelo nome; nomes iguais para conteúdos distintos exigem consulta por ObjectID. `metadata_chunk_descriptor` e `metadata_chunk_peers`

alimentam `directory_lookup`; este consulta `superpeer_find_member` para transformar NodeIDs em endpoints IP/porta. `metadata_remove_peer` remove disponibilidades quando há LEAVE voluntário.

5. Fluxo completo do upload

1. Preparação: `peer.c:main` interpreta `upload` e `local_control_command` envia o pedido por socket Unix ao Peer ativo. `local_control.c:handle_command` cria uma `FileSession` com o NodeID do serviço e chama `file_client_upload`.
2. Identificação: `content_validate_pdf` aceita nome com extensão `.pdf` (sem analisar a assinatura interna). `object_id_file` calcula o SHA-256 integral e o tamanho. `file_client_upload` calcula a quantidade de chunks e marca LZ4.
3. Início: o serviço descobre o NodeID do Peer destino com PING e envia `STORE/BEGIN`. `peer.c:handle_store` chama `storage_begin`, que cria ou retoma um registro em pending.
4. Chunks em paralelo: `transfer_worker_count` escolhe até 8 workers por padrão, limitado por CPUs e número de chunks; `PEER_TRANSFER_THREADS` permite 1-32. Cada `upload_worker` usa `pread_all` no offset próprio, calcula SHA-256, comprime com LZ4 e envia `STORE/CHUNK` em conexão própria.
5. Recepção: `protocol_receive_message` valida o CRC32. `storage_put_chunk` descomprime, recalcula o SHA-256, confere hash/índice/offset/tamanho e grava chunk e manifest em pending. O Peer responde ACK somente após sucesso.
6. Commit: `file_client_upload` envia `STORE/COMMIT`. `storage_commit` chama `verify_document`: verifica os hashes de todos os chunks, tamanho e ObjectID completo. Depois publica o objeto em `objects`.
7. Catálogo: `peer.c:announce_document` envia descritores e localização ao Super Peer. `register_announcement` valida o membro e chama `directory_announce/metadata_announce`. O upload só exibe sucesso depois do ACK desse anúncio.

Saída esperada: File, Size, ObjectID, Chunks, um hash por Chunk, Compression: LZ4 e Upload completed. Os hashes exibidos são do conteúdo original de cada chunk, não dos bytes comprimidos.

```
arquivo.pdf -> ObjectID -> chunks -> SHA-256 por chunk -> LZ4
-> STORE/BEGIN -> STORE/CHUNK -> STORE/COMMIT
-> ANNOUNCE -> índice do Super Peer
```

6. Fluxo completo do download

1. A CLI encaminha download ao Peer ativo via `local_control_command`; o serviço chama `file_client_download`.
2. `lookup_document` descobre a identidade do Super Peer e envia LOOKUP por nome ou ObjectID. `superpeer.c:answer_lookup` chama `directory_lookup` e devolve documento, descritores e endpoints de cada chunk.
3. O Peer cria `<destino>.part` com exclusividade e pré-aloca o tamanho. Os `download_worker` repartem índices; `download_one` envia `DOWNLOAD_REQ` diretamente ao Peer que possui cada chunk.
4. `peer.c:handle_download` chama `storage_read_chunk` e responde com `DOWNLOAD_REP`. O receptor valida CRC32, ObjectID, índice, offset, tamanho, descompressão e SHA-256 contra o hash recebido e o descritor retornado pelo LOOKUP. Se falhar, tenta o próximo endpoint disponível para o chunk.
5. `pwrite_all` grava cada chunk no offset correto. Depois de todos os workers, `object_id_file` recalcula o SHA-256 do `.part` e confere tamanho e ObjectID.
6. `fsync`, link e remoção do `.part` publicam o destino sem sobrescrever arquivo existente. Só então aparecem Download completed e SHA-256 verified.

```
LOOKUP -> descritores + Peers -> DOWNLOAD_REQ por chunk
-> CRC32 -> LZ4 -> SHA-256 por chunk -> .part
-> SHA-256 do arquivo -> publicação -> destino.pdf
```

7. Armazenamento, integridade e recuperação

Cada Peer usa `.peer_storage/<porta>/pending/<objectid>/` para uploads incompletos e `.peer_storage/<porta>/objects/<objectid>/` para documentos concluídos. O manifest versionado registra documento, proprietário, estado e descritores. Caminhos internos usam ObjectID hexadecimal; o nome enviado pelo usuário não vira caminho do catálogo.

Existem três conferências diferentes: CRC32 detecta alteração do frame TCP; SHA-256 do chunk detecta conteúdo recebido ou armazenado incorretamente após descompressão; ObjectID confere o documento completo. A validação no download compara também o hash do índice recebido por LOOKUP. Falhas não publicam um destino final, e outro Peer pode ser tentado para o chunk.

Escritas de chunks e manifests usam temporários e fsync; o commit publica o diretório concluído por rename. Um BEGIN compatível pode retomar dados em pending, e novo COMMIT do mesmo ObjectID é tratado de modo idempotente. Ao reiniciar, `storage_create` carrega manifests válidos; `peer_service_run` faz JOIN, chama `announce_catalog` e repubblica documentos finais verificados. Reiniciar somente o Super Peer enquanto os Peers continuam ativos não dispara reanúncio automático.

8. Escolhas de implementação

- OpenSSL EVP, zlib e LZ4 prontos: o código usa bibliotecas existentes para SHA-256, CRC32 e compressão, sem reimplementar algoritmos.
- Chunks de 4 MiB: permitem hashes e tentativas por parte, paralelismo e retomada; LZ4 por chunk permite descompressão independente.
- `pread/pwrite` e conexão por worker: threads usam offsets explícitos e não compartilham posição do arquivo nem estado do framing TCP.
- `pending`, `objects` e `.part`: separam dados incompletos de dados publicados; link impede substituir destino existente.
- Hash table local e cópias: estrutura simples para localizar documentos; mutex e cópias evitam expor ponteiros internos alteráveis por outra thread.
- ANNOUNCE após COMMIT: o índice só aponta para um objeto que o Peer terminou de verificar e publicar. A resposta de sucesso ao usuário exige ACK do catálogo.
- Identidade persistente e canal Unix: a CLI usa o NodeID do Peer ativo; o UUID salvo evita trocar a identidade a cada reinício.
- Serialização explícita: structs com ponteiros e padding ficam locais; o wire tem tamanhos e ordem de bytes definidos.

9. Funções para explicar na apresentação

Aluno 1: `file_client_upload` coordena preparação, BEGIN, workers e COMMIT; `upload_worker` prepara e envia um chunk; `storage_put_chunk` valida e persiste; `storage_commit/verify_document` publicam após hash final; `file_client_download` coordena LOOKUP e destino; `download_one` valida, tenta endpoints e escreve; `protocol_receive_message` protege o frame com CRC32.

Aluno 2: `node_compute_id` cria o NodeID; `superpeer_register_node` mantém membros; `object_id_file` cria o ObjectID; `metadata_announce` valida e publica metadados/localizações; `metadata_find_document`, `metadata_chunk_peers` e `metadata_chunk_descriptor` consultam o índice; `directory_announce` e

directory_lookup ligam rede, metadados e membros; register_announcement/answer_lookup atendem os comandos TCP.

Explicação em uma frase: “O aluno 1 move, armazena e verifica os bytes; o aluno 2 cria o identificador e mantém o índice que diz quais Peers possuem cada chunk.”

10. Demonstração e verificações

Inicie em terminais separados, com as portas livres:

```
make all
./bin/superpeer --port 55101 --name superpeer
./bin/peer serve 55102 127.0.0.1 55101
./bin/peer serve 55103 127.0.0.1 55101
```

Depois, em outro terminal na pasta do PDF:

```
./bin/peer upload arquivo.pdf 127.0.0.1 55102
./bin/peer --local-peer-port 55103 download arquivo.pdf recebido.pdf 127.0.0.1 55101
cmp arquivo.pdf recebido.pdf
```

Sem --local-peer-port, o executor local padrão é o Peer 55102. No exemplo, o Peer 55103 executa o download, consulta o Super Peer e busca os chunks do Peer 55102. cmp compara os arquivos byte a byte.

Verificações disponíveis: make test-aluno2 cobre APIs locais de identidade, membros e metadados; make test-c2 cobre protocolo/armazenamento; python3 tests/c2/integration.py --bin-dir bin cobre fluxo TCP. Nesta preparação do guia, make test-aluno2 test-c2 BIN_DIR=/tmp/c2-guide-bin compilou e passou os testes locais do aluno 2, mas o teste de protocolo parou dentro do sandbox porque o ambiente bloqueou operações com sockets locais. A suíte make test-c2 BIN_DIR=/tmp/c2-guide-bin foi repetida fora do sandbox e passou: transfer/storage negative tests: ok e storage atomic/fault tests: ok. A integração TCP não foi repetida nesta preparação. O relatório histórico refatoracao_checkpoint_2.md registra 20/20 no roteiro C2 e 46 verificações na integração, em execução anterior.

11. Limites reais do checkpoint

O Super Peer usa índice em memória; não há persistência ou replicação desse índice. LEAVE voluntário remove o membro e suas localizações, mas ainda não há heartbeat ou detecção automática de queda. A aceitação de PDF verifica a extensão, não o formato interno. CRC32 e SHA-256 detectam corrupção, mas o protocolo atual não autentica criptograficamente os Peers. Chord, Gossip, eleição, replicação automática, consenso, cache LFU, 2PC e transferência incremental pertencem a checkpoints posteriores. A API local de metadados e o fluxo TCP integrado são verificações diferentes; testes executados cobrem somente os cenários que realmente rodam.

Fontes no repositório

Código: peer.c, file_client.c, storage.c, node.c, superpeer.c, metadata.c, directory.c, protocol.c, transfer_protocol.c. Especificação e histórico: requisitos_trabalho.md, requisitos_aluno_2.md, refatoracao_checkpoint_2.md, protocolo_checkpoint_2.md e explicacao_codigo_aluno_1.md / explicacao_codigo_aluno_2.md.